

# DP Computer Science: A 3.3.3 Updating Data in Databases with SQL

---

## 1. Core Concepts & Learning Objectives

### Learning Intentions

By the end of this lesson, students will be able to:

- Explain the purpose and syntax of the SQL **INSERT INTO** statement for adding new records
  - Explain the purpose and syntax of the SQL **UPDATE SET** statement for modifying existing data
  - Explain the purpose and syntax of the SQL **DELETE** statement for removing records
  - Describe the potential **performance implications** of using the UPDATE statement on indexed columns
  - Discuss the concept of **index rebuilding or reorganisation** following significant data modifications
  - Apply **INSERT, UPDATE, and DELETE** statements in a practical database environment (DB Browser for SQLite)
- 

### Key Vocabulary

| Term                   | Definition                                                             |
|------------------------|------------------------------------------------------------------------|
| INSERT INTO            | SQL command used to add new records (rows) to a table                  |
| UPDATE SET             | SQL command used to modify existing records in a table                 |
| DELETE                 | SQL command used to remove records from a table                        |
| WHERE clause           | Specifies which rows should be affected by an UPDATE or DELETE         |
| Index                  | A database structure that speeds up data retrieval (like a book index) |
| Index overhead         | The additional work required to maintain indexes when data is modified |
| Index rebuild          | The process of reorganising an index to improve performance            |
| Autocommit             | When changes are automatically saved to the database                   |
| Foreign key constraint | A rule that ensures data integrity between related tables              |

# Relevant ATL Skills

| Skill           | Description                                                                                               |
|-----------------|-----------------------------------------------------------------------------------------------------------|
| Thinking        | Critical thinking (analysing impact of SQL commands);<br>evaluation (considering efficiency implications) |
| Communication   | Explaining concepts; using appropriate SQL terminology                                                    |
| Self-Management | Organisation; self-directed learning                                                                      |

## 2. Introduction to DML (Data Manipulation Language)

### What is DML?

DML (Data Manipulation Language) is the set of SQL commands used to modify the actual data stored in a database.

### The Three Core DML Commands

text

DML COMMANDS

INSERT INTO  
Adds new records to a table

UPDATE SET  
Modifies existing records

DELETE FROM  
Removes records from a table

# Comparison of DML Commands

| Command     | Purpose                 | Impact       | Risk                     |
|-------------|-------------------------|--------------|--------------------------|
| INSERT INTO | Add new records         | Adds data    | Minimal (if careful)     |
| UPDATE SET  | Modify existing records | Changes data | Can affect many rows     |
| DELETE FROM | Remove records          | Removes data | Permanently removes data |

## 3. INSERT INTO

### Purpose

The `INSERT INTO` statement adds new records (rows) to a database table.

### Basic Syntax

```
sql

INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

### Examples

Example 1: Inserting into specific columns

```
sql

INSERT INTO Students (StudentID, FirstName, LastName, GradeLevel)
VALUES (101, 'Alice', 'Smith', 11);
```

Example 2: Inserting into all columns

```
sql

INSERT INTO Students
VALUES (102, 'Bob', 'Jones', '1990-05-15', 10, TRUE);
```

Example 3: Inserting multiple records

sql

```
INSERT INTO Students (StudentID, FirstName, LastName, GradeLevel)
VALUES
    (103, 'Charlie', 'Brown', 9),
    (104, 'Diana', 'Williams', 12),
    (105, 'Eve', 'Davis', 10);
```

## 4. UPDATE SET

### Purpose

The **UPDATE SET** statement modifies existing records in a database table. The **WHERE** clause specifies which records to update.

### Basic Syntax

sql

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

### The Importance of the WHERE Clause

text

#### WHY THE WHERE CLAUSE IS CRITICAL

```
UPDATE Students
SET GradeLevel = 12;
```

⚠ NO WHERE CLAUSE!  
ALL students will be set to GradeLevel 12!

```
UPDATE Students
SET GradeLevel = 12
WHERE StudentID = 101;
```

✅ WHERE CLAUSE INCLUDED!

Only the student with StudentID 101 is updated!

## Examples

### Example 1: Updating a single record

```
sql

UPDATE Students
SET GradeLevel = 12
WHERE StudentID = 101;
```

### Example 2: Updating multiple columns

```
sql

UPDATE Students
SET GradeLevel = 11, LastName = 'Smith-Jones'
WHERE StudentID = 102;
```

### Example 3: Updating multiple records

```
sql

UPDATE Students
SET GradeLevel = GradeLevel + 1
WHERE GradeLevel >= 9;
```

### Example 4: Using conditions in UPDATE

```
sql

UPDATE Students
SET IsEnrolled = FALSE
WHERE GradeLevel > 12 OR GradeLevel < 1;
```

---

## 5. DELETE

## Purpose

The `DELETE FROM` statement removes records from a database table. The `WHERE` clause specifies which records to delete.

## Basic Syntax

```
sql

DELETE FROM table_name
WHERE condition;
```

## The Importance of the WHERE Clause

text

### WHY THE WHERE CLAUSE IS CRITICAL

```
DELETE FROM Students;
```



NO WHERE CLAUSE!

ALL records will be deleted from the Students table!

This is often irreversible!

```
DELETE FROM Students
```

```
WHERE StudentID = 101;
```



WHERE CLAUSE INCLUDED!

Only the student with StudentID 101 is deleted!

## Examples

### Example 1: Deleting a single record

```
sql

DELETE FROM Students
```

```
WHERE StudentID = 101;
```

### Example 2: Deleting multiple records

```
sql

DELETE FROM Students
WHERE GradeLevel < 9;
```

### Example 3: Deleting with multiple conditions

```
sql

DELETE FROM Students
WHERE GradeLevel > 12
AND IsEnrolled = FALSE;
```

## Cautions for DELETE

| Caution          | Description                                                                    |
|------------------|--------------------------------------------------------------------------------|
| Irreversible     | Deleted data is often permanently removed                                      |
| Foreign Keys     | May violate foreign key constraints                                            |
| Backups          | Ensure a backup exists before deleting data                                    |
| Always Use WHERE | Never use DELETE without a WHERE clause (unless you intend to delete all rows) |

## 6. Performance Implications of Indexes on Updates

### What is an Index?

An index is a database structure that speeds up data retrieval. Like the index of a book, it allows the database to find data quickly without scanning the entire table.

### Index Analogy

text

INDEX ANALOGY

BOOK INDEX

Apples ..... 12, 45, 78

Bananas ..... 23, 56, 89

Cherries ..... 34, 67, 90

Just like a book index helps you find pages quickly, a database index helps the database find rows quickly.

✔

Speeds up SELECT queries

✘

Slows down INSERT, UPDATE, DELETE (must update index)

## How Indexes Affect Data Modifications

| Command | Impact on Index                                        | Performance Effect                     |
|---------|--------------------------------------------------------|----------------------------------------|
| INSERT  | New entry must be added to index                       | Slower inserts (index must be updated) |
| UPDATE  | If indexed column changes, index entry must be updated | Slower updates on indexed columns      |
| DELETE  | Index entry must be removed                            | Slower deletes (index must be updated) |
| SELECT  | Index speeds up searches                               | Much faster searches                   |

## Example: Updating an Indexed Column

```
sql

-- Suppose there is an index on LastName
UPDATE Students
SET LastName = 'Anderson'
WHERE StudentID = 101;
```



What happens:

- 1. The database finds the row (fast, using the index)
- 2. The row is updated
- 3. The index entry must be **updated** (slower)
- 4. The index is reorganised to maintain order

Why Indexes Slow Down Modifications

| Reason               | Description                                                   |
|----------------------|---------------------------------------------------------------|
| Maintenance Overhead | The index must be updated whenever the indexed column changes |
| Reorganisation       | Indexes must be reorganised to maintain sorted order          |
| Space Allocation     | New index entries require space; may cause page splits        |
| I/O Operations       | More disk operations are required to update indexes           |

7. Index Rebuilding and Reorganisation

Why Indexes Need Rebuilding

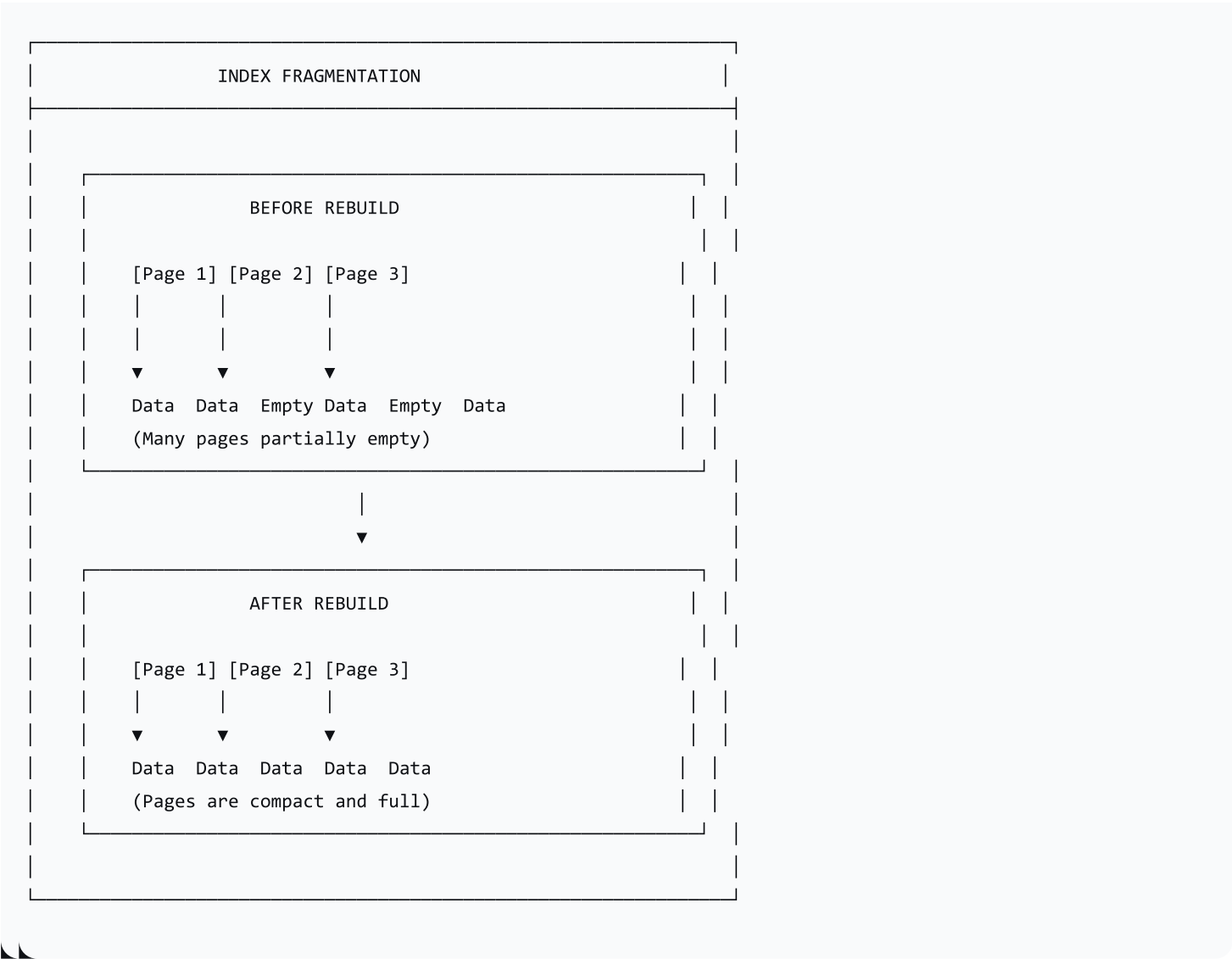
Over time, indexes can become fragmented or inefficient due to repeated INSERT, UPDATE, and DELETE operations. Rebuilding reorganises the index to restore performance.

When to Rebuild Indexes

| Scenario                       | Reason                                      |
|--------------------------------|---------------------------------------------|
| After large data modifications | Many INSERT, UPDATE, or DELETE operations   |
| Fragmented index               | Index pages are split; performance degrades |
| Data migration                 | Moving large amounts of data                |
| Regular maintenance            | Scheduled index maintenance                 |

Understanding Index Fragmentation

text



## Index Rebuild Process

| Step                      | Description                             |
|---------------------------|-----------------------------------------|
| 1. Identify fragmentation | The database detects fragmented indexes |
| 2. Create new index       | A new, compact index is created         |
| 3. Drop old index         | The old fragmented index is removed     |
| 4. Use new index          | The new index is used for queries       |

## 8. SQL Command Summary

### INSERT INTO

sql

```
-- Insert one record
INSERT INTO Students (StudentID, FirstName, LastName, GradeLevel)
VALUES (106, 'Frank', 'Miller', 11);

-- Insert multiple records
INSERT INTO Students (StudentID, FirstName, LastName, GradeLevel)
VALUES
    (107, 'Grace', 'Lee', 10),
    (108, 'Henry', 'King', 9);
```

## UPDATE SET

sql

```
-- Update a single record
UPDATE Students
SET GradeLevel = 12
WHERE StudentID = 101;

-- Update multiple records
UPDATE Students
SET GradeLevel = GradeLevel + 1
WHERE GradeLevel >= 9 AND GradeLevel < 12;

-- Update with multiple conditions
UPDATE Students
SET IsEnrolled = FALSE, GraduationYear = 2026
WHERE GradeLevel = 12;
```

## DELETE FROM

sql

```
-- Delete a single record
DELETE FROM Students
WHERE StudentID = 101;

-- Delete multiple records
DELETE FROM Students
WHERE GradeLevel > 12;

-- Delete with multiple conditions
```

```
DELETE FROM Students
WHERE GradeLevel < 9 AND IsEnrolled = FALSE;
```

## 9. Lesson Sequence

### Lesson Plan (60 minutes)

| Phase                  | Time   | Activity                                                 |
|------------------------|--------|----------------------------------------------------------|
| Introduction           | 5 min  | Welcome; lesson objectives; overview of DML commands     |
| Introduction to DML    | 5 min  | Role of SQL and DML; three main commands                 |
| INSERT INTO            | 5 min  | Purpose and syntax; examples of inserting data           |
| UPDATE SET             | 10 min | Purpose and syntax; importance of WHERE clause; examples |
| Performance of Indexes | 5 min  | What is an index?; performance impact on writes          |
| Index Rebuilding       | 5 min  | Why indexes need rebuilding; fragmentation               |
| DELETE                 | 5 min  | Purpose and syntax; emphasis on WHERE clause; examples   |
| Practical Activity     | 15 min | Students work on worksheet in DB Browser for SQLite      |
| Review                 | 5 min  | Summary; questions; address common challenges            |

### Practical Activity: Worksheet Tasks

| Task   | Description                                         |
|--------|-----------------------------------------------------|
| Task 1 | Insert a new student record into the Students table |

| Task   | Description                                |
|--------|--------------------------------------------|
| Task 2 | Update a student's grade level             |
| Task 3 | Update multiple students' grade levels     |
| Task 4 | Delete a student record                    |
| Task 5 | Delete all students below a certain grade  |
| Task 6 | Observe the importance of the WHERE clause |

## Observation Activity: Index Performance

| Observation   | Description                                                  |
|---------------|--------------------------------------------------------------|
| Before Update | Observe the speed of SELECT queries with and without indexes |
| During Update | Observe the time taken to update indexed columns             |
| After Rebuild | Observe the effect of index rebuilding on performance        |

## 10. Assessment Activities

| Assessment Type     | Description                                                  |
|---------------------|--------------------------------------------------------------|
| Class Participation | Observe engagement and contributions                         |
| Worksheet           | Evaluate ability to use INSERT, UPDATE, and DELETE           |
| Exit Ticket         | Gauge understanding and identify areas needing clarification |

### Sample Exit Ticket Questions:

1. What is the purpose of the INSERT INTO statement?
2. Why is the WHERE clause important in UPDATE and DELETE statements?
3. What happens if you use UPDATE without a WHERE clause?
4. Why does updating an indexed column slow down the database?

# 11. Differentiation

| Level     | Strategy                                                                                       |
|-----------|------------------------------------------------------------------------------------------------|
| Support   | Provide scaffolded queries; work through first examples together; one-on-one support           |
| Challenge | Challenge with complex UPDATE statements with multiple conditions; research index optimisation |

# 12. Resources

- Presentation (Slide Deck)
- eBook (A3.3.3 with examples)
- Worksheet: INSERT, UPDATE, DELETE exercises
- Worksheet Answers
- Indexing Activity
- DB Browser for SQLite

# 13. Summary: Key Takeaways

| Command        | Purpose                               | Key Point                                |
|----------------|---------------------------------------|------------------------------------------|
| INSERT INTO    | Add new records                       | Add one or many records at a time        |
| UPDATE SET     | Modify existing records               | Always use WHERE clause!                 |
| DELETE FROM    | Remove records                        | Always use WHERE clause!                 |
| Index Overhead | Updates on indexed columns are slower | Indexes speed SELECT but slow writes     |
| Index Rebuild  | Reorganises fragmented indexes        | Improves performance after large updates |

# Key Reminders

text

#### CRITICAL REMINDERS

- ⚠ ALWAYS use a WHERE clause in UPDATE and DELETE
- ⚠ Without WHERE, you modify/delete ALL rows
- ⚠ Deleted data is often unrecoverable
- ⚠ Updating indexed columns is slower
- ⚠ Back up data before performing large updates/deletes

*"The power to modify data comes with great responsibility—always use the WHERE clause and understand the performance implications of your actions."*